HANDWRITTEN DIGITS RECOGNITION

Complete Project Explanation & Technical Documentation

Capstone 1: PRCP-1002 – Handwritten Digits Image Processing

A practical guide to the MNIST dataset, exploratory data analysis, preprocessing, machine-learning models, evaluation metrics, results interpretation, limitations, and professional GitHub presentation.

Prepared as a learning and project-reference document

How to use this document

This document explains the project from the problem statement through evaluation and presentation. It is written as a companion to your Jupyter notebook. The notebook remains the source of truth for the exact code, charts, trained-model outputs, and measured scores.

Important: Some numerical details below are based on the notebook information previously shared. Where a result depends on the improved notebook being executed, this guide explains what to report rather than inventing a new score.

Contents

1. Project overview and objective
2. Dataset: MNIST
3. End-to-end workflow
4. Tools and libraries
5. Exploratory data analysis (EDA)
6. Image preprocessing
7. Machine-learning algorithms
8. Training, validation, and test strategy
9. Evaluation metrics
10. Understanding project outputs and charts
11. Historical results and fair-comparison warning
12. Limitations and future improvements
13. GitHub repository structure
14. How to explain the project in an interview
15. Common interview questions
16. Final submission checklist

1. Project overview and objective

The Handwritten Digits Recognition project is an image-classification task. Its goal is to take an image containing one handwritten numeric digit and predict which class it belongs to: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9.

Input: a 28 × 28 grayscale image. **Output:** one predicted digit class from 10 possible classes.

The project uses the well-known MNIST benchmark dataset. It explores the data, prepares images for learning algorithms, trains several models, compares their performance, and examines the types of errors they make.

Why is this useful?

Digit recognition is a building block for systems that read numeric information from scanned documents. Related applications include cheque processing, postal-code reading, form digitization, and automated reading of examination or registration forms. This notebook is a benchmark experiment; it should not be described as a production banking or postal system.

Project objective

- Inspect and understand the image dataset before modelling.
- Prepare image pixels in formats appropriate for different algorithms.
- Train and compare K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Multi-Layer Perceptron (MLP), and Convolutional Neural Network (CNN).
- Evaluate predictions using more than accuracy alone.
- Study confusion matrices and misclassified images to understand model weaknesses.

2. Dataset: MNIST

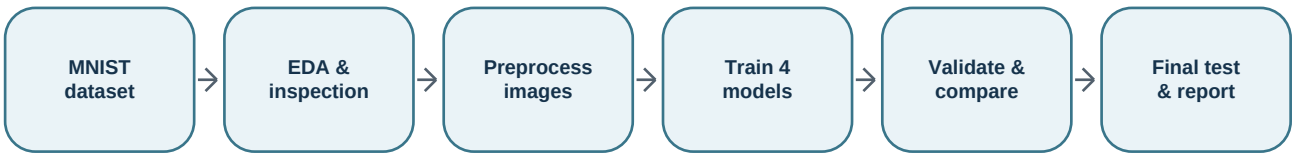
MNIST stands for Modified National Institute of Standards and Technology. It is a standard dataset used to teach and benchmark handwritten digit classification.

Property	Description
Total images	70,000
Training split	60,000 images
Official test split	10,000 images
Image dimensions	28 × 28 pixels
Colour format	Grayscale
Classes	10 digit labels: 0–9
Pixel values	Unsigned 8-bit integers, typically 0–255
Features per flattened image	28 × 28 = 784 pixel values

A grayscale image stores brightness rather than three colour channels. Each pixel is a number. In the usual MNIST display, 0 represents black and 255 represents white, with values between them representing shades of gray. The image is therefore a small 28-row by 28-column grid of numbers.

For example, a single image can be represented as a 28 × 28 matrix. A classical model such as KNN or SVM usually receives that matrix flattened into a vector with 784 values. A CNN can keep the two-dimensional spatial arrangement.

3. End-to-end workflow



The workflow begins with data inspection, not model training. EDA helps reveal the image shape, label distribution, pixel behaviour, and possible data-quality issues. Preprocessing then converts the raw images into model-ready inputs. Models are trained and compared using a consistent evaluation design. Finally, results are interpreted and documented.

4. Tools and libraries

Tool / library	Purpose in this project
Python	Main programming language for data handling and model development.
Jupyter Notebook	Interactive environment for code, charts, outputs, and explanations.
NumPy	Efficient numerical arrays and image-matrix operations.
Pandas	Tabular summaries, label counts, and structured data inspection.
Matplotlib	Displaying digit images, distributions, curves, and other plots.
Scikit-learn	Classical ML models, preprocessing pipelines, metrics, and evaluation.
TensorFlow / Keras	Building and training the CNN model.

5. Exploratory data analysis (EDA)

EDA means examining and visualizing data before selecting or training a model. It helps answer: What does one record look like? Are labels present? Are there missing values? Are classes balanced? What do pixel values look like?

5.1 Dataset dimensions and data types

The notebook reports 60,000 training images and 10,000 test images, each with a 28 × 28 shape. The image arrays use uint8 values. Confirm the actual shapes in the notebook output if you have changed the data-loading code.

5.2 Pixel range and summary statistics

The previously shared notebook output reported pixel values from 0 to 255, a mean pixel intensity around 33.32, and a standard deviation around 78.57. These values describe the pixel distribution, not model accuracy. The low mean is consistent with many dark background pixels and relatively fewer bright digit strokes.

5.3 Missing values and label classes

The notebook inspection reported no missing values in the image arrays and ten digit classes. A useful class-count chart should show the number of examples for every digit. If the counts are close, the dataset is approximately balanced; quantify the largest and smallest class counts rather than relying only on visual appearance.

5.4 Sample digit images

A grid of example images allows a human to check that labels match visible digits and shows natural handwriting variation. Include examples from all ten classes. If some digits are hard to distinguish, describe the visual ambiguity rather than assuming the model is wrong in every case.

5.5 Mean image for each class

A mean image is formed by averaging the pixel values of all images belonging to one digit class, pixel by pixel. It reveals the typical stroke pattern for that class. Similar mean images can indicate why some pairs of digits are confused. However, averaging can blur handwriting variations and does not represent a real handwritten sample.

5.6 Pixel-intensity distribution

A histogram shows how often different pixel brightness values occur. If a large proportion of pixels are near zero, that indicates that the image background occupies much of the 28 × 28 canvas. The previously shared output estimated about 80.9% of pixels as zero/background pixels. Use the exact calculation and definition in your notebook when reporting this percentage.

5.7 Additional quality checks

- Check for duplicate images and, where possible, duplicate image-label pairs.
- Inspect samples from every digit class, not only a few convenient examples.
- Display difficult or ambiguous examples and compare their true labels with predictions.
- Report class counts and percentages to support any claim about class balance.
- Place a short interpretation immediately below each chart: what is visible, what it means, and why it matters for modelling.

6. Image preprocessing

Preprocessing converts raw pixel arrays into a form that algorithms can use reliably. Keep the preprocessing steps consistent between training, validation, and test data.

6.1 Normalization

A common step is dividing every pixel by 255. This maps the original 0–255 range to approximately 0–1. For example, a pixel value of 128 becomes 128/255 ≈ 0.502. Neural networks often train more smoothly when input values are on a small, consistent scale.

```
X_normalized = X.astype('float32') / 255.0
```

6.2 Flattening for classical models

KNN, SVM, and a standard dense MLP generally expect each image as a one-dimensional feature vector. Flattening converts 28 × 28 into 784 features. The pixel order must remain the same for every image.

```
X_flat = X_normalized.reshape(number_of_images, 784)
```

6.3 Keeping image structure for CNN

A CNN uses local spatial patterns, so its input should preserve height and width. For a grayscale image, a channel dimension is added: 28 × 28 × 1. This allows convolution filters to examine nearby pixels and learn edges and strokes.

```
X_cnn = X_normalized.reshape(number_of_images, 28, 28, 1)
```

6.4 Standardization for classical estimators

StandardScaler transforms a feature using the training-set mean and standard deviation. It is often used in an Scikit-learn Pipeline with models such as SVM or MLP. Fit the scaler only on the training data; the pipeline then applies the learned transformation to validation and test data. This prevents information leakage.

7. Machine-learning algorithms

The project compares four approaches. They learn in different ways, so their strengths and weaknesses are not identical.

7.1 K-Nearest Neighbors (KNN)

KNN is an instance-based method. It stores the training examples. To classify a new image, it calculates the distance from that image to stored training images, selects the k closest examples, and predicts the most common label among those neighbours.

Example: if k = 5 and the five nearest training images have labels [3, 3, 3, 8, 5], KNN predicts 3 because it receives three votes.

Important setting: k controls how many neighbours vote. A very small k can be sensitive to noise; a larger k can smooth decisions but may blur class boundaries. KNN can be computationally expensive at prediction time because it compares a new sample with many stored examples.

7.2 Support Vector Machine (SVM) with RBF kernel

An SVM learns a decision boundary that separates classes. The Radial Basis Function (RBF) kernel lets it form nonlinear boundaries by comparing examples through a similarity function. This is useful when digit classes cannot be separated by a simple straight boundary.

C parameter: controls the penalty for classification errors. A larger C places more emphasis on fitting the training data; too large a value can overfit. A smaller C allows a wider margin and more training errors. The RBF gamma parameter controls how local each training example's influence is.

7.3 Multi-Layer Perceptron (MLP / ANN)

An MLP is a feed-forward neural network made of layers of connected neurons. A flattened image supplies 784 input features. Hidden dense layers learn combinations of pixel patterns, and an output layer produces scores for ten digit classes. Softmax converts output scores into class probabilities.

During training, the network predicts labels, calculates a loss, and uses backpropagation with an optimizer to adjust weights. Hidden-layer sizes and training settings affect learning capacity and overfitting.

7.4 Convolutional Neural Network (CNN)

A CNN is designed for grid-like data such as images. Convolution filters slide across the image and learn local patterns such as edges, curves, and strokes. Deeper layers combine these into more complex digit features.

CNN component	What it does
Convolution	Applies learnable filters to detect local image patterns.
ReLU activation	Adds non-linearity by setting negative activations to zero.
Pooling	Reduces spatial dimensions and retains salient information.
Flatten	Converts feature maps into a vector for dense layers.
Dense output layer	Produces scores for the ten digit classes.
Softmax	Converts scores into a probability distribution.
Dropout	Randomly disables some units during training to reduce overfitting.
Early stopping	Stops training when validation performance stops improving, according to the chosen rule.

The CNN often performs strongly on image tasks because it preserves spatial relationships and learns useful visual filters. It also typically requires more compute and careful training.

8. Training, validation, and test strategy

A reliable evaluation separates model fitting, model selection, and final performance measurement.

Data portion	Purpose	How it should be used
Training set	Learn model parameters	Fit models and preprocessing transformations.
Validation set	Choose model settings	Compare candidates and select settings without using the final test set.
Test set	Estimate final generalization	Use only after model selection is complete.

A fair model comparison requires the same data conditions. The original notebook used approximately 15,000 training examples for the classical models but approximately 54,000 for the CNN. Because the CNN had access to substantially more training data, the original scores are not a strictly controlled comparison.

The improved evaluation design described in the earlier project discussion uses the same stratified 15,000-example training subset for all four models and a common 6,000-example validation set. The official 10,000-example test set is held back for final evaluation. A stratified split aims to preserve the class proportions across subsets.

Model settings should be selected using validation performance, such as macro-F1. After choosing a model and configuration, evaluate the selected approach on the untouched test set. Do not repeatedly tune against the test set, because doing so leaks information from the test data into model selection.

The previously described candidate settings included KNN neighbour counts of 3, 5, and 7; SVM RBF C values of 2, 5, and 10; and MLP layouts of (128) and (128, 64). The CNN used a dropout/early-stopping approach. Confirm the actual executed notebook cells before reporting exact configurations.

9. Evaluation metrics

Accuracy is useful, but it can hide class-specific weaknesses. Precision, recall, F1-score, macro averages, and a confusion matrix provide a fuller view.

9.1 Accuracy

Accuracy is the proportion of all predictions that are correct.

```
Accuracy = Correct predictions / Total predictions
```

9.2 Precision

For a given class, precision answers: of all samples predicted as this class, how many truly belong to it? Low precision means many false positives for that class.

```
Precision = TP / (TP + FP)
```

9.3 Recall

Recall answers: of all samples that truly belong to this class, how many did the model correctly find? Low recall means many false negatives.

```
Recall = TP / (TP + FN)
```

9.4 F1-score and macro-F1

F1 is the harmonic mean of precision and recall. It is high only when both are reasonably high. Macro-F1 calculates F1 separately for each class and then averages the ten class scores equally. This gives each digit equal weight, even if class counts differ.

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
Macro-F1 = (F1_0 + F1_1 + ... + F1_9) / 10
```

9.5 Confusion matrix

A confusion matrix compares actual labels with predicted labels. In a typical plot, rows represent actual classes and columns represent predicted classes. Strong diagonal values mean correct predictions. Off-diagonal values show which digits are confused with other digits. Always check the axis labels because plotting libraries can use different conventions.

9.6 Training and prediction time

Training time measures how long model fitting takes under the recorded hardware and software conditions. Prediction time measures the time needed to classify a specified number of samples. State the sample count and environment when comparing timings; timings can change substantially across machines.

10. Understanding project outputs and charts

Output / chart	How to interpret it
Sample digit grid	Confirms the images look like handwritten digits and demonstrates writing variation.
Class distribution	Shows the number of examples for each label; report actual counts and percentages.
Mean image per digit	Shows the average stroke pattern for each class; similar patterns may help explain confusion.
Pixel-intensity histogram	Shows how frequently brightness values occur; background pixels often dominate.
Confusion matrix	Shows correct classifications on the diagonal and specific class-to-class errors elsewhere.
CNN accuracy curve	Shows training and validation accuracy across epochs; a widening gap can indicate overfitting.
CNN loss curve	Shows training and validation loss; validation loss rising while training loss falls is a warning sign.
Misclassified examples	Shows difficult images and helps explain errors such as unusual writing, tilted strokes, or ambiguous shapes.

For every chart in the notebook, add three short points: (1) what the chart displays, (2) the main observation supported by the plot, and (3) what that observation means for the project. Avoid statements that are not supported by the displayed data.

11. Historical results and fair-comparison warning

The following accuracy values were previously visible in the original notebook discussion. They are historical results from the original experiment, not verified results from the improved controlled comparison.

Model	Historical original accuracy
KNN	95.25%
SVM (RBF)	97.22%
MLP / ANN	95.68%
CNN	98.90%

Because the original CNN was trained on a larger subset than the classical models, these values should not be presented as a perfectly fair head-to-head ranking. Do not copy these scores into a new results table unless they match the executed notebook output and are clearly labelled as original historical results.

For the improved notebook, populate a fresh results table using the actual run outputs. Include training size, validation size, selected settings, accuracy, macro-precision, macro-recall, macro-F1, and timing. Report final test metrics only for the selected model or models after model selection is complete.

12. Limitations and future improvements

- MNIST images are small, centered, grayscale digits; real-world scanned documents may contain noise, skew, varied backgrounds, and different writing styles.
- A limited training subset may not capture the full variety in the dataset.
- Hyperparameter exploration may be limited; a wider search or cross-validation could provide a more robust comparison.
- Results depend on random seeds, library versions, hardware, and exact preprocessing.
- The notebook is an experimental benchmark and does not by itself provide a deployed application or production monitoring.

Future improvements

- Use data augmentation such as small rotations, shifts, and zooms, while ensuring labels remain valid.
- Explore more CNN architectures and tune learning rate, batch size, dropout, and filter counts.
- Use stratified cross-validation for more stable classical-model comparisons when compute allows.
- Test on a separate external handwritten-digit dataset to assess performance beyond MNIST.
- Add confidence scores, calibration checks, and a review path for uncertain predictions.
- Build a small web interface or API, then measure latency, memory use, and model size.

13. GitHub repository structure

A clean repository separates the notebook, documentation, supporting images, and environment information. Keep generated charts in an images folder so the README can display them.

```
PRCP-1002-Handwritten-Digits-Recognition/
■■■ README.md
■■■ notebooks/
■   ■■■ PRCP_1002_Handwritten_Digits_Recognition.ipynb
■■■ images/
■   ■■■ mnist_samples.png
■   ■■■ class_distribution.png
■   ■■■ pixel_distribution.png
■   ■■■ confusion_matrix.png
■   ■■■ cnn_accuracy.png
■   ■■■ cnn_loss.png
■   ■■■ prediction_examples.png
■■■ requirements.txt
■■■ .gitignore
```

README recommendations

- Start with the project title and a concise one-paragraph summary.
- List dataset details, tools, algorithms, and evaluation approach.
- Explain how to install dependencies and open/run the notebook.
- Embed a few meaningful charts and prediction examples.
- Include results copied from the actual notebook run, with the training/validation/test protocol.
- Document limitations and future work. Never claim deployment or business use unless you have actually built and tested it.

Example requirements file

```
numpy
pandas
matplotlib
scikit-learn
tensorflow
jupyter
```

These are indicative package names. If your notebook uses additional packages or particular versions, record the actual environment with pip freeze or a carefully maintained requirements file. Do not claim that this example list is guaranteed to reproduce every environment.

14. How to explain the project in an interview

“I worked on a handwritten digit recognition project using the MNIST dataset. The dataset contains 28 × 28 grayscale images representing digits from 0 to 9. I first performed exploratory data analysis to inspect image dimensions, pixel ranges, class distribution, sample images, mean digit patterns, and pixel intensities. I then normalized the pixel values and prepared two input formats: flattened 784-feature vectors for classical models and 28 × 28 × 1 images for the CNN. I compared KNN, RBF SVM, MLP, and CNN using classification metrics such as accuracy, precision, recall, macro-F1, and confusion matrices. I also inspected misclassified examples to understand the model’s errors. An important

evaluation improvement was to use consistent training and validation data for fair comparison and reserve the official test set for final evaluation.”

If asked what you personally did, describe only the tasks you actually completed. For example: data inspection, preprocessing, model training, evaluation, chart interpretation, and documentation—provided you performed those steps yourself.

15. Common interview questions

What is MNIST?

A benchmark dataset of 70,000 grayscale handwritten digit images, each 28 × 28 pixels, split into 60,000 training and 10,000 test images.

Why normalize pixel values?

To scale values from 0–255 to 0–1, which often helps numerical stability and neural-network training.

Why flatten images for KNN and SVM?

These classical estimators generally expect a two-dimensional samples-by-features matrix; flattening makes each image a 784-feature row.

Why does CNN keep the 2D shape?

Convolutions exploit local spatial relationships and learn image patterns such as edges and strokes.

What is overfitting?

When a model learns training-specific details and performs worse on unseen data; compare training and validation behaviour to detect it.

Why use a validation set?

To choose model settings without using the final test set.

Why is test-set leakage a problem?

Repeatedly using test results to tune choices makes the test score optimistic and no longer an independent estimate.

What does macro-F1 tell you?

It gives each class equal weight by averaging per-class F1 scores.

What does a confusion matrix show?

Which classes are correctly predicted and which pairs of classes are confused.

Why might 4 and 9 be confused?

Some handwriting styles have similar shapes; inspect the actual misclassified examples before stating which pair is most common.

Why can KNN be slow at prediction?

It may need to calculate distances to many stored training examples for each new image.

What is the role of dropout?

It randomly disables a fraction of units during training to reduce reliance on particular activations and help control overfitting.

16. Final submission checklist

- Notebook runs from top to bottom without errors.
- Notebook contains outputs for important cells and clearly labelled charts.
- Data source and split sizes are documented.
- Preprocessing is applied consistently and fitted only on training data where applicable.
- Model comparisons use the same training and validation data.
- The official test set is not used repeatedly for model selection.
- Results tables are copied from the actual executed notebook.
- Every chart has a short, evidence-based interpretation.
- Confusion matrix axes and metric averaging methods are clearly identified.
- README, requirements file, notebook, and images are organized professionally.
- Repository contains no passwords, API keys, private data, or unnecessary large generated files.

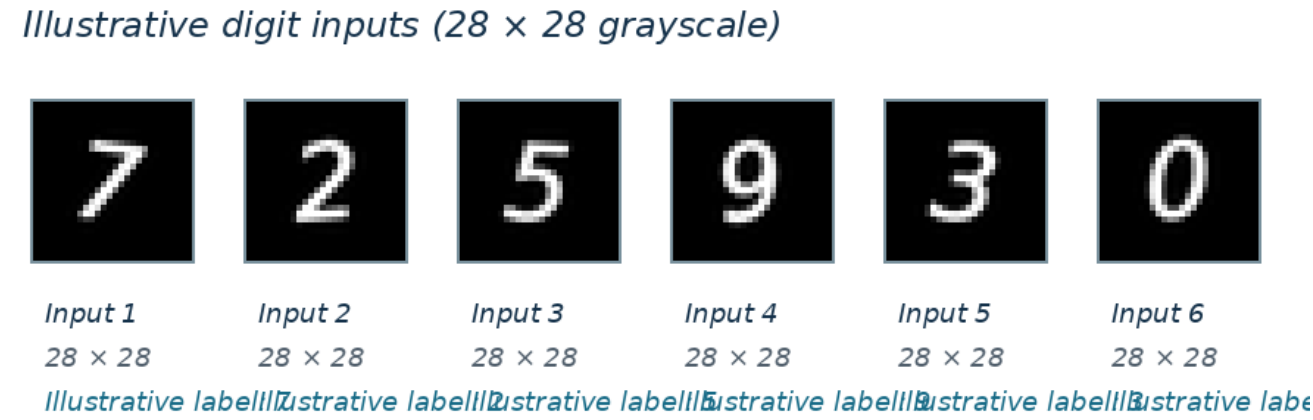
Closing note

The strongest version of this project is not simply the one with the highest accuracy. It is the one with a clear problem definition, sound data handling, fair evaluation, honest interpretation of results, and reproducible documentation. Use this guide alongside your actual notebook outputs, and update any configuration or score that changes when you rerun the notebook.

VISUAL APPENDIX: INPUTS, OUTPUTS & WORKING IMAGES

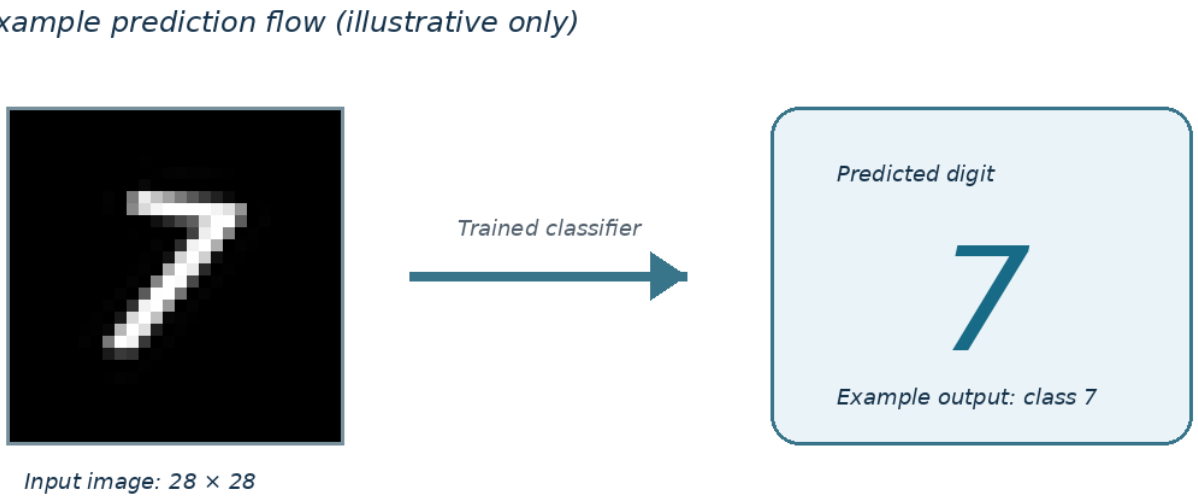
This appendix adds visual examples to the project explanation. The digit pictures below are synthetic illustrations created to demonstrate the input format and processing flow. They are **not extracted from the MNIST dataset, not screenshots from your notebook, and not predictions generated by your trained model**. Replace them with your own notebook output images for a verified project demonstration.

1. Example input images



Each tile represents a single-digit grayscale input resized to 28 × 28 pixels. In the real MNIST dataset, the image is stored as a 28-by-28 array of pixel intensities. The small images are enlarged using nearest-neighbour scaling so individual pixels are visible.

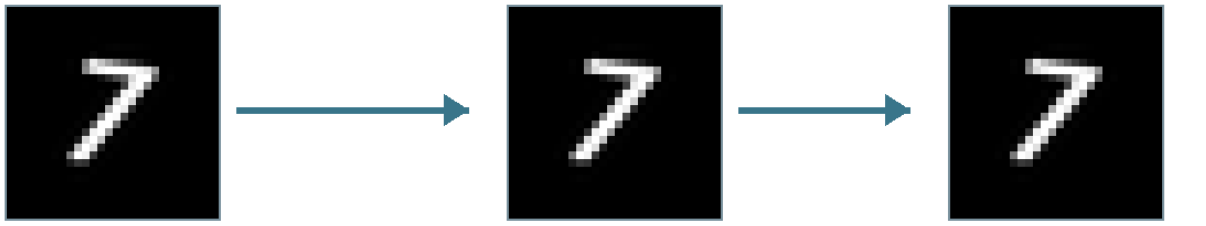
2. Example input → model → output



Illustrative example: the input image depicts a 7 and the example output is class 7. This diagram explains the expected interface between an input image and a classifier. It does not prove that the project model achieved this prediction. For your final report, capture a real prediction from your executed notebook and insert its image here.

3. Image preprocessing stages

How one digit image is prepared for a CNN



1. Raw grayscale image
28 × 28 pixels
values: 0 to 255

2. Normalized image
pixel ÷ 255
values: 0 to 1

3. CNN input tensor
28 × 28 × 1
height × width × channel

The same image can be represented in three ways: (1) raw grayscale pixels, usually integers in the range 0–255; (2) normalized pixels after dividing by 255, usually floating-point values in the range 0–1; and (3) a CNN tensor with shape 28 × 28 × 1. The drawing shows the conceptual stages; the normalized view is visually similar because normalization changes numeric scale, not the intended appearance.

4. End-to-end working flow

Project workflow: from image to predicted digit



The notebook loads the dataset, inspects examples and labels, preprocesses the images, trains one or more classifiers, predicts labels for unseen examples, and evaluates predictions using metrics such as accuracy, precision, recall, macro-F1, and a confusion matrix.

5. How to add genuine working screenshots from your notebook

• **Step 1 — Show a real input:** display a sample image from the MNIST test set with its true label.

• **Step 2 — Show a real prediction:** run the selected model on that image and display the predicted digit beside the true label.

• **Step 3 — Show model evidence:** include the confusion matrix and, for the CNN, training/validation accuracy and loss curves.

• **Step 4 — Save the plots:** use Matplotlib's savefig, for example `plt.savefig('images/prediction_example.png', dpi=200, bbox_inches='tight')`, then add the saved images to the GitHub repository.

• **Step 5 — Be precise:** record the model name and actual result from the run. Do not use illustrative pictures as evidence of model accuracy.

Example notebook code for a genuine prediction

Use this as a pattern and adjust variable names to match your notebook. It assumes `X_test` is normalized, `y_test` contains the true labels, and `model` is your already-trained classifier.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
index = 0
image = X_test[index]
true_label = int(y_test[index])

# Adapt this section to your model's expected input shape
prediction = model.predict(image.reshape(1, 28, 28, 1))
predicted_label = int(np.argmax(prediction, axis=1)[0])

plt.imshow(image.squeeze(), cmap='gray')
plt.title(f'True: {true_label} | Predicted: {predicted_label}')
plt.axis('off')
plt.savefig('images/prediction_example.png', dpi=200, bbox_inches='tight')
plt.show()
```

For KNN, SVM, or an MLP that expects flattened input, use the corresponding 784-feature row rather than the CNN tensor. If your model was trained through a preprocessing pipeline, pass data in the same input format that the pipeline expects.

Recommended real screenshots for the final GitHub README

1. A grid of at least 10 real MNIST test images with true labels.
2. A prediction panel showing true and predicted labels for several test examples.
3. The selected model's confusion matrix.
4. CNN training and validation curves, if the CNN is included.
5. A compact results table copied from the executed notebook.